Module: CSCI369 Ethical Hacking
Assignment: Assignment Q5
UOW Student number: 1103891
Full name: Jon Tan
Tutorial group: T03F

(1) Port discovery:

1. Using the **netdiscover** command '*netdiscover -i eth0 -r 192.168.122.0/24*', we are able to discover and list all ip and mac on the network.
    - **-i** defines the interface
    - **-r** defines the range

[Kali]

```
Currently scanning: Finished!   |   Screen View: Unique Hosts

1 Captured ARP Req/Rep packets, from 1 hosts.   Total size: 42
_____________________________________________________________________________
  IP            At MAC Address     Count     Len  MAC Vendor / Hostname
_____________________________________________________________________________
192.168.122.10  52:54:00:dd:db:0d      1      42  Unknown vendor
```

List shows an IP of 192.168.122.10, which is our Ubuntu VM running the executable_server.

2. Next we will use **nmap** and run '*nmap -sU -p 12345-12500 192.168.122.10*' to scan for ports between 12345 and 12500.
    - **-sU** tells nmap to perform an UDP scan
    - **-p** defines the ports

[Kali] From the screenshot below, we can observe that the only open port is on port **12404**.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -sU -p 12345-12500 -v 192.168.122.10
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-22 05:29 -0500
Initiating ARP Ping Scan at 05:29
Scanning 192.168.122.10 [1 port]
Completed ARP Ping Scan at 05:29, 0.03s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 05:29
Completed Parallel DNS resolution of 1 host. at 05:29, 0.50s elapsed
Initiating UDP Scan at
Scanning 192.168.122.10
Increasing send delay f
crease to 4
Increasing send delay f
increase to 5
Increasing send delay f
 increase to 6
Increasing send delay f
 increase to 7
Increasing send delay f
 increase to 8
Discovered open port 12404/udp on 192.168.122.10
UDP Scan Timing: About 29.23% done; ETC: 05:31 (0:01:15 remaining)
Increasing send delay for 192.168.122.10 from 800 to 1000 due to 11 out of 28 droppe
d probes since last increase.
UDP Scan Timing: About 55.71% done; ETC: 05:31 (0:00:58 remaining)
UDP Scan Timing: About 74.94% done; ETC: 05:31 (0:00:34 remaining)
Completed UDP Scan at 05:31, 151.97s elapsed (156 total ports)
Nmap scan report for 192.168.122.10
Host is up (0.00045s latency).
Not shown: 155 closed udp ports (port-unreach)
PORT      STATE SERVICE
12404/udp open  unknown
MAC Address: 52:54:00:DD:DB:0D (QEMU virtual NIC)

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 152.57 seconds
           Raw packets sent: 257 (7.331KB) | Rcvd: 159 (9.021KB)
```

```
Completed UDP Scan at 05:31, 151.97s elapsed (156 total ports)
Nmap scan report for 192.168.122.10
Host is up (0.00045s latency).
Not shown: 155 closed udp ports (port-unreach)
PORT      STATE SERVICE
12404/udp open  unknown
MAC Address: 52:54:00:DD:DB:0D (QEMU virtual NIC)
```

Discovered open port

[Ubuntu] On the server we can see the response that executable_server has replied to kali

```
Waiting for a message...
Sent response to ('192.168.122.193', 65219).
```

(2) Obtaining voucher code:

[Kali] With the open port (**12404**) we found previously in step (1), we will attempt connection with the Ubuntu UDP server using netcat.

This is the netcat command ran: '*nc -u 192.168.122.10 12404*'

- **-u** defines that it is in UDP mode
- **12404** was the port we scanned for in step (1)

Next, I sent over my 7-digit UOW student number as the client ID. In this case, it is **1103891** (Turquoise box).

The server replies back with a hashed voucher code:
fbf47eb2a1cc1863b122f6c74262c4f5 (Red box)

```
┌──(root㉿kali)-[/home/kali/Documents/CSCI369/python/Assignment/Q5]
└─# nc -u 192.168.122.10 12404
1103891
Your client ID, 1103891, has been transformed into a secret! Here is your voucher co
de: fbf47eb2a1cc1863b122f6c74262c4f5
```

[Kali] I created a file called 'VoucherCode.txt' that stores the hashed voucher code generated previously.

```
┌──(root㉿kali)-[/home/kali/Documents/CSCI369/python/Assignment/Q5]
└─# echo "fbf47eb2a1cc1863b122f6c74262c4f5" > VoucherCode.txt

┌──(root㉿kali)-[/home/kali/Documents/CSCI369/python/Assignment/Q5]
└─# ls
A_candidate.txt  B_candidate.txt  VoucherCode.txt

┌──(root㉿kali)-[/home/kali/Documents/CSCI369/python/Assignment/Q5]
└─# cat VoucherCode.txt
fbf47eb2a1cc1863b122f6c74262c4f5
```

(3) Cracking A & B in voucher code:

The given formula for the VoucherCode is: MD5(*A*||ClientID||*B*),
Where A is two-alphabet characters and B is two-symbol characters

The candidate space for A & B respectively:

- A: 26^2
- B: $symbol_set^2$

Open a new terminal and generate a word list for the hashcat cracking later on.

We will be using Crunch to generate the candidate spaces as per required by the assignment brief.

What is Crunch?
Crunch is a tool that generates password candidates based on rules you give it, like: fixed length (e.g. 2 characters), character sets(e.g. Uppercase letters, symbols)

Crunch does not do the cracking of hashes, it simply enumerates possible inputs.

[Kali] **Generate A||ClientID||B possibilities**
Crunch command ran:
'crunch 11 11 -t @@1103891^^ -o A-ClientID-B.txt'

Generated a dictionary file containing words with a minimum and maximum length of 11 (11 11) using the given characters
(@@ [wild card for lower/uppercase characters],
1103891 [ClientID, this is 7 characters],
^^ [wild card for special symbols]),
saving the output to a file (-o A-ClientID-B.txt):

```
┌──(root㉿kali)-[/home/kali/Documents/CSCI369/python/Assignment/Q5]
└─# crunch 11 11 -t @@1103891^^ -o A-ClientID-B.txt
Crunch will now generate the following amount of data: 8833968 bytes
8 MB
0 GB
0 TB
0 PB
Crunch will now generate the following number of lines: 736164

crunch: 100% completed generating output

┌──(root㉿kali)-[/home/kali/Documents/CSCI369/python/Assignment/Q5]
└─# cat A-ClientID-B.txt
aa1103891!!
aa1103891!@
aa1103891!#
aa1103891!$
aa1103891!%
aa1103891!^
aa1103891!&
aa1103891!*
aa1103891!(
aa1103891!)
aa1103891!-
```

We can afford to create the dictionary list this way by putting the ClientID in the middle because we know that this value will never change. We also know that A is always 2 alphabet long and B is always 2 symbols long.

Also, in hashcat it doesn't allow for a dictionary || mask || dictionary input. The closest option, which was -a 1 (Combination), only allows parsing of 2 dictionary files. Therefore this option was considered.

[Kali] **Hashcat cracked**
Command ran:
'hashcat -m 0 -a 0 VoucherCode.txt A-ClientID-B.txt'

- -m 0 defines MD5 hashing algorithm
- -a 0 defines straight attack mode. Only requires a word list consisting of the potential passwords.

Highlighted in a red box is the cracked A||ClientID||B input.

```
Dictionary cache built:
* Filename..: A-ClientID-B.txt
* Passwords.: 736164
* Bytes.....: 8833968
* Keyspace..: 736164
* Runtime...: 0 secs

fbf47eb2a1cc1863b122f6c74262c4f5:kz1103891`}

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: fbf47eb2a1cc1863b122f6c74262c4f5
Time.Started.....: Sun Feb 22 10:08:47 2026 (0 secs)
Time.Estimated...: Sun Feb 22 10:08:47 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (A-ClientID-B.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:  6574.2 kH/s (0.14ms) @ Accel:1024 Loops:1 Thr:1 Vec:16
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (ne
w)
Progress.........: 311296/736164 (42.29%)
Rejected.........: 0/311296 (0.00%)
Restore.Point....: 309248/736164 (42.01%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: kx1103891 ^ → kz1103891,&

Started: Sun Feb 22 10:08:37 2026
Stopped: Sun Feb 22 10:08:49 2026
```

Therefore, we can deduce that:

- **A = kz**
- **B = `}**

[Kali] **VoucherCode proof**
To prove that kz1103891`} is the original input, I will hash this input once again with MD5 to compare the outputs.

I have written a simple python code that takes in the VoucherCode and hashes it with the MD5 algorithm.

We can see that the hash output matches the voucher code provided by the server. This shows that

- **A = kz**
- **B = `}**

```
┌──(root㉿kali)-[/home/kali/Documents/CSCI369/python/Assignm
└─# cat MD5_hash.py
import hashlib

VoucherCode = input("Enter your VoucherCode for hashing: ")

hash = hashlib.md5(b'kz1103891`}').hexdigest()

print(hash)

┌──(root㉿kali)-[/home/kali/Documents/CSCI369/python/Assignm
└─# python3 MD5_hash.py
Enter your VoucherCode for hashing: kz1103891`}
fbf47eb2a1cc1863b122f6c74262c4f5
```

```
┌──(root㉿kali)-[/home/kali/Documents/CSCI369/python/Assignment/Q5]
└─# nc -u 192.168.122.10 12404
1103891
Your client ID, 1103891, has been transformed into a secret! Here is your voucher co
de: fbf47eb2a1cc1863b122f6c74262c4f5
```